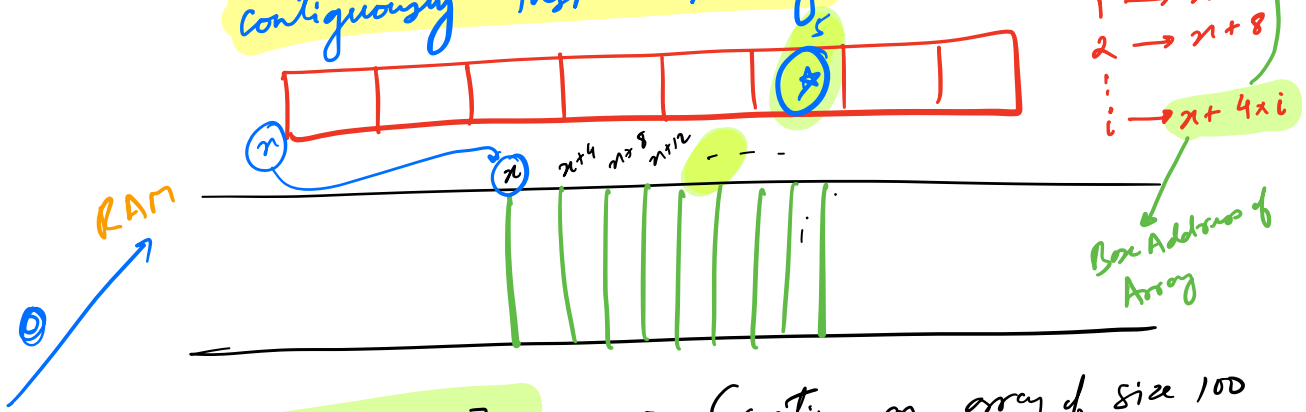
Arrays

Linear D.S. → Logically linear

Collection of same datatype elements, stored contiguously inside memory!



`int A[100];` → Creating an array of size 100

- datatype
- name
- Size

10^3 B → 1 KB

10^3 KB → 1 MB

10^6 B → 1 MB

1 int → 4B

10^6 int → 4×10^6 B → 4 MB

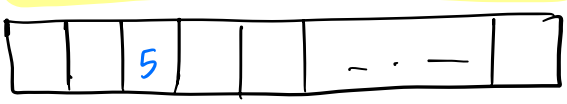
ML : 256MB

$1 <= N <= 10^6$

`int A[N];`

Indexing

idx → 0 1 2 3 4 . — N-1



N

O(1)

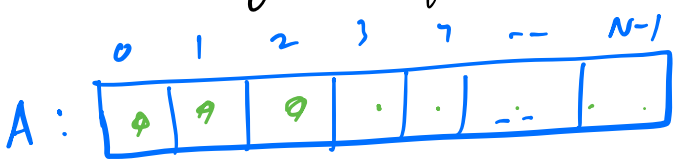
A[2] = 5

↳ update

print(A[N-1]);

↳ print the last element!

1. Given an Array A of size N. Print it!




```
print(A[0]);
print(A[1]);
print(A[2]);
.
.
print(A[N-1]);
```

```
for(i = 0; i < N; i++){
    print(A[i]);
}
```

Q Given N elements → Array.
Count the no. of elements having atleast 1 element greater than itself!

A = [-3, -2, 6, 8, 4, 8, 5] → 5

A: [0, 2, 5, 1, 4, 8, 1, 3, 8] → 7

A: [2, 3, 10, 7, 3, 2, 10, 8] → 6

1) SORT INC order → O(N lg N)

+

find

2 2 3 3 7 8 10 10 → O(N)

+

2)

3) ANS = N − Count(MAX(A)) → O(1)

$N <= 10^6$

N lg N + N + 1

TC = O(N lg N) ✓

STEPS:

1. Find the MAX element
2. Count the freq of the MAX element.
3. ANS = N − Count(MAX)

$1 <= N <= 10^6$

$-10^9 <= A[i] <= 10^9$

1. $-\infty$ — -10^9-1

A : [−3, −2, 6, 8, 4, −5, 8, 7]

mC = −∞ → −3 → −2 → 6 → 8 → 8 → 8 → 8 → 8

```
int mC = -10^9 - 1;
for(i=0; i<N; i++) {
    if(A[i] > mC){
        mC = A[i];
    }
}
int cnt = 0;
for(i=0; i<N; i++){
    if(A[i] == mC){
        cnt++;
    }
}
ret N-cnt;
```

N x O(1) + N x O(1)

Constraints

$1 <= N <= 10^6$

$-10^9 <= A[i] <= 10^9$

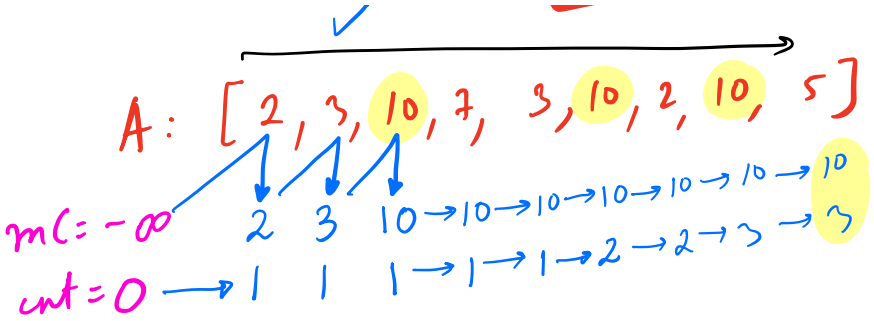
-2×10^9 INT 2×10^9 (32)

-9×10^{18} LONG 9×10^{18} (64)

TC = O(N)

SC = O(1)

II)




```
mC = -∞, cnt = 0;
for(i = 0; i < N; i++) {
    if( A[i] > mC) {
        mC = A[i];
        cnt = 1;
    }
    else if( A[i] == mC){
        cnt++;
    }
}
out N - cnt;
```

N x O(1)

TC = O(N)

Q Given an array of size N & k.

Check if there exists a pair (i, j)

: $A[i] + A[j] = K$ && $(i != j)$

2-SUM

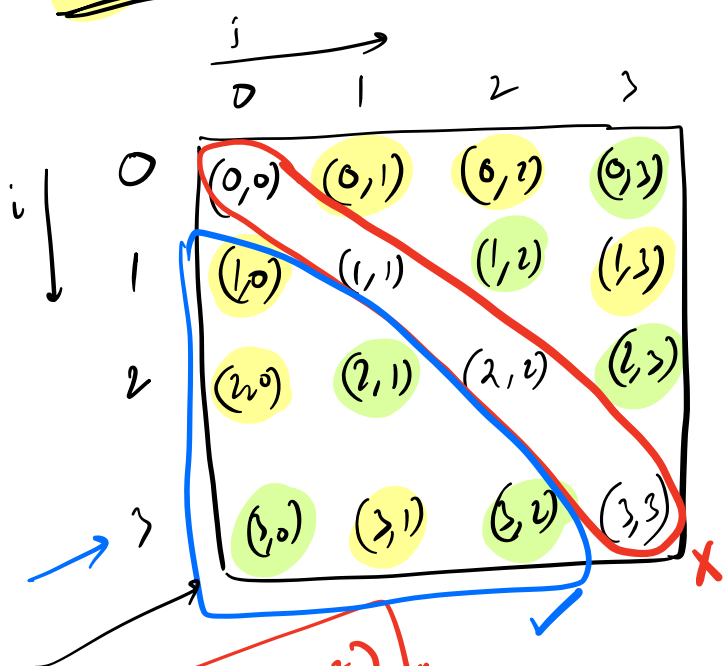
A : [3, -2, 1, 4, 3, 6, 8]

0 1 2 3 4 5 6

K = 10 → true

K = 8 → false

N = 4 [_____]



TC = O(N²)

SC = O(1)

$A_i + A_j == K$

$A_j + A_i == K$

```
N
for(i=0; i<N; i++){
    for(j=0; j<N; j++){
        if(i!=j){
            if(A[i]+A[j]
               == K){
                ret true;
            }
        }
    }
}
ret false;
```

N

O(1)

i = 0 | j = x

i = 1 | j = 0

i = 2 | j = 0, 1

i = 3 | j = 0, 1, 2

i: 1 → N-1

i j: 0 → i-1

j < i

```
for (i=0; i<N; i++) {
    for (j=0; j<i; j++) {
        if (A[i] + A[j] == K) {
            ret true;
        }
    }
}
ret false;
```

i	j: [0-i-1]	# it
0	x	0
1	[0-0]	+ 1
2	[0-1]	+ 2
⋮	⋮	⋮
N-1	[0-N-2]	N-1

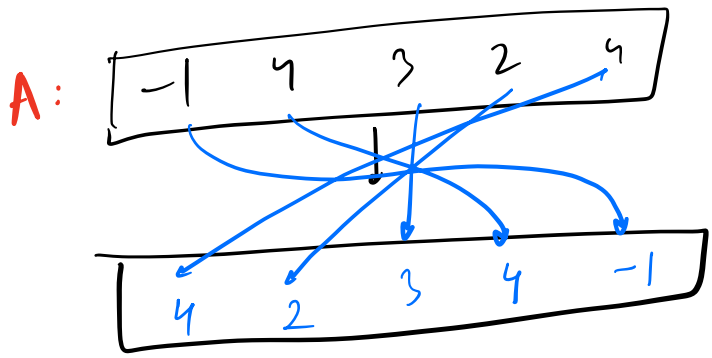
$TC = O(N^2)$

$\frac{N^2 - N}{2}$

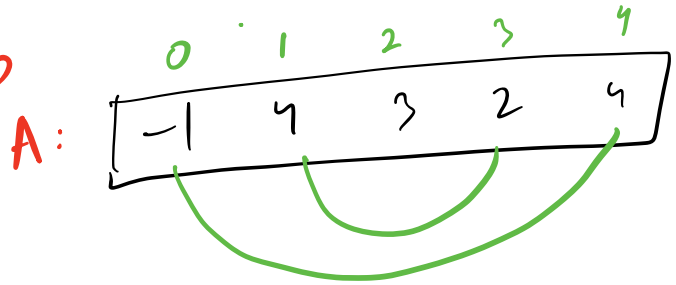
$\frac{(N-1)N}{2}$

Given an Array. Reverse it INPLACE!

↓ IN the same Array!



ODD

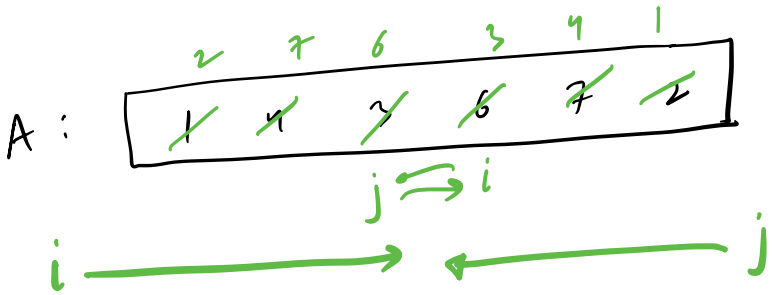


STOP

i == j

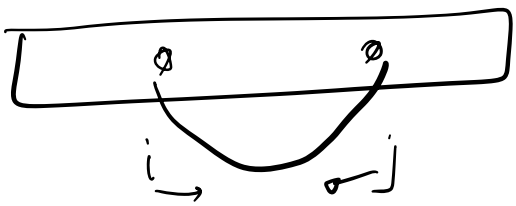
i > j

EVEN



```
int i = 0 , j = N-1
while( i<j ){
    swap( A[i], A[j]);
    i++;
    j--;
}
```

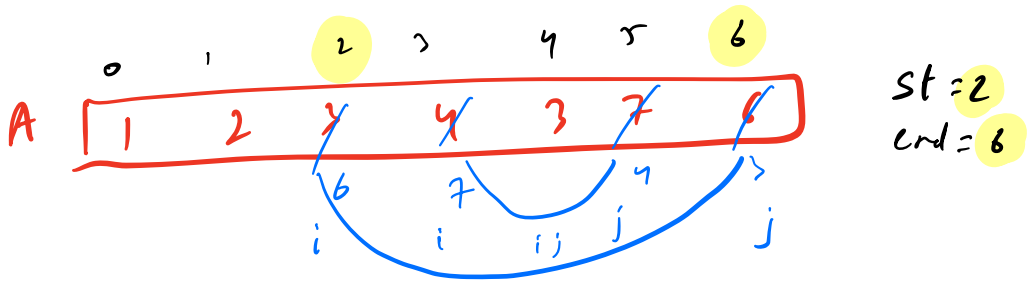
$N/2 \times O(1)$



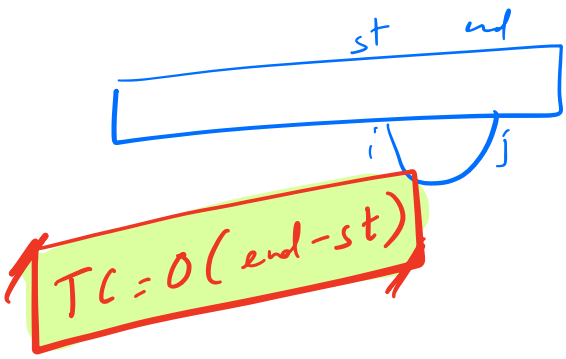
$TC = O(N)$

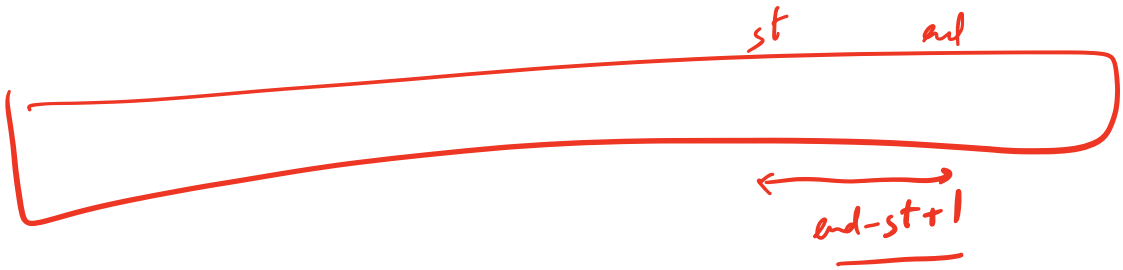
$SC = O(1)$

Q Given an Array & st & end indexes of a sub-array! Reverse that subarray!

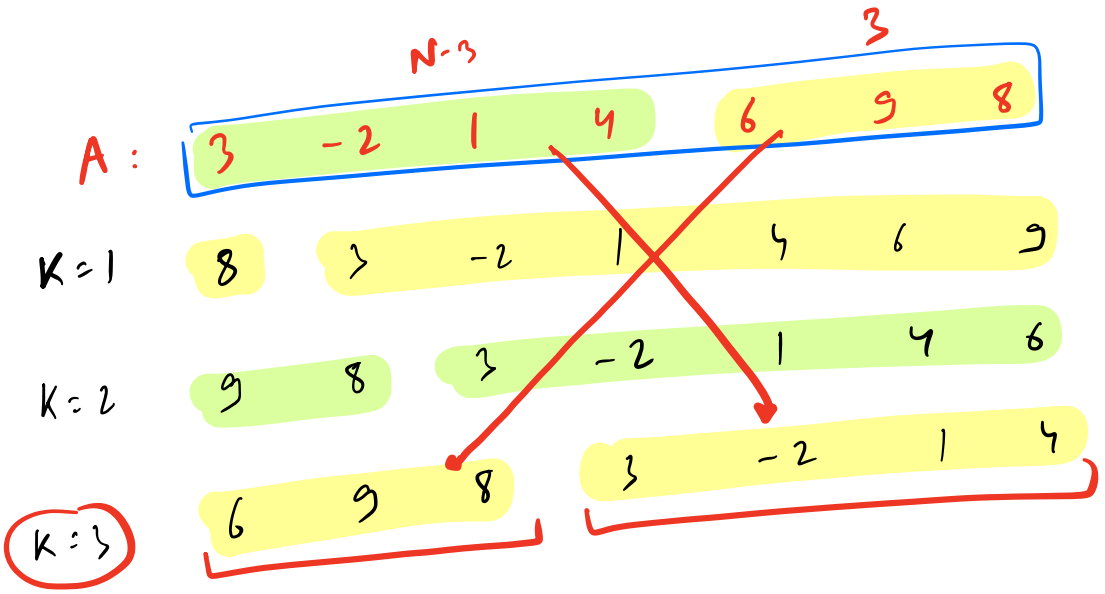



```
void reverse ( A[], st, end) {
    int i = st , j = end
    while( i<j) {
        swap( A[i], A[j]);
        i++;
        j--;
    }
}
```



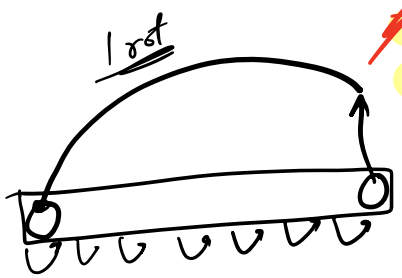


Q Given an array.
Rotate the array from last to first K times



I) BF

1. Write the code for rotating 1 time
2. Call K times! → K x O(N)

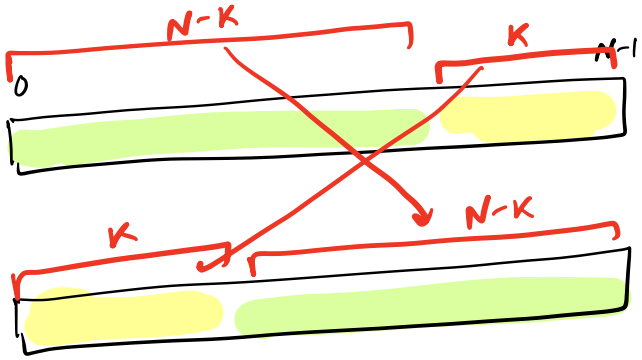


TC = O(NK)

$1 <= N <= 10^6$

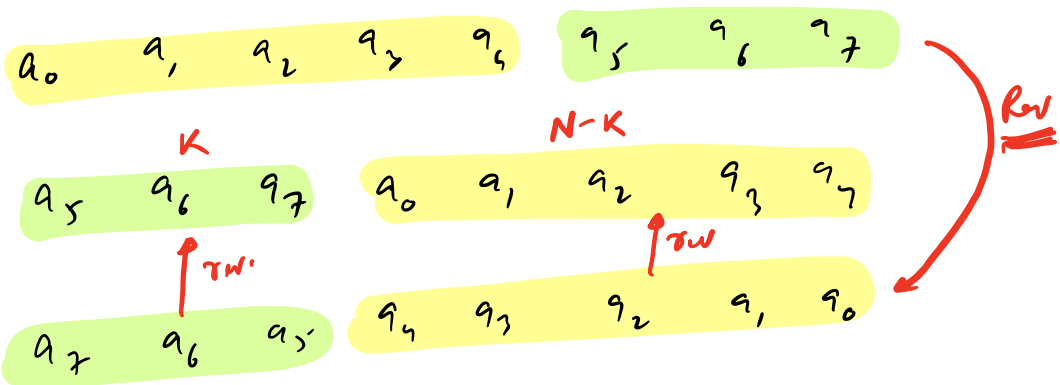
$1 <= K <= 10^9$

II



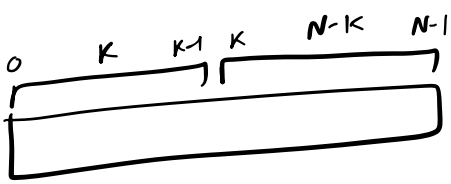
After K rot

A:



STEPS:

1. Reverse the whole Array
2. Reverse first K elements
3. —— last N-K ——

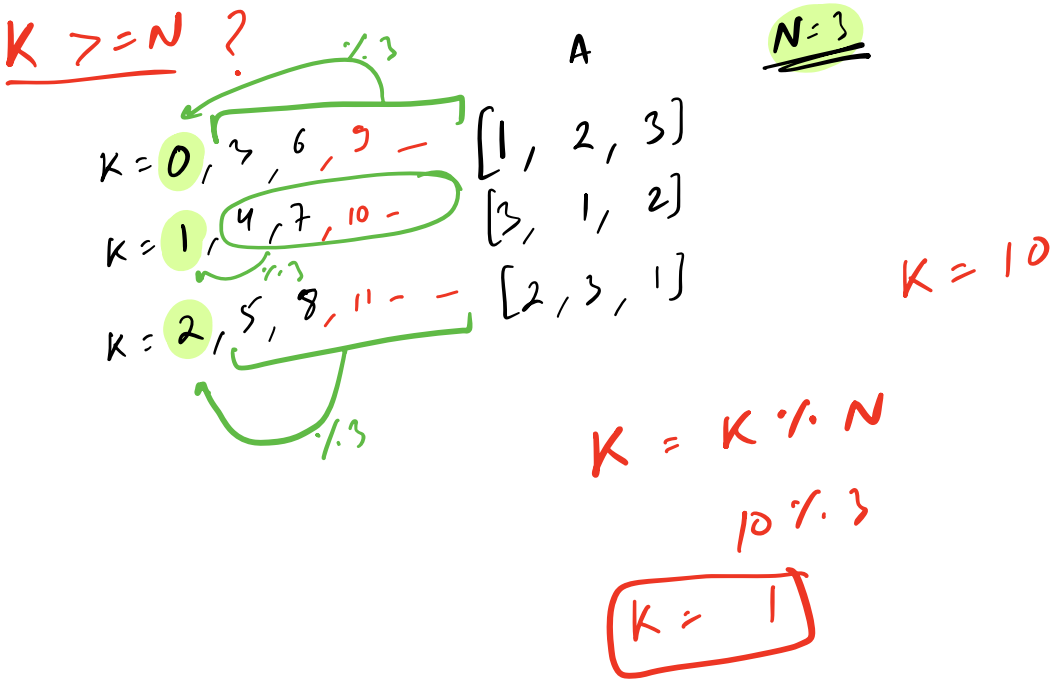


✓ reverse(A, 0, N-1); → N
+
✓ ——, 0, K-1); → K
+
✓ ——, K, N-1); → N-K

2N

SC = O(1)

TC = O(N)


K >= N ?
A
N = 3
K = 0, 3, 6, 9 -
K = 1, 4, 7, 10 -
K = 2, 5, 8, 11 - -
[1, 2, 3]
[3, 1, 2]
[2, 3, 1]
%3
%3
%3
K = 10
K = K % N
10 % 3
K = 1